Bioinformatics 2024-25

HELLENIC REPUBLIC
National and Kapodistrian University of Athens
EST. 1837

Department of Informatics

GATK Variant Calling Analysis

Vasileios-Klearchos Chatzitolios

Student Number: 7115152400026

Abstract

This report describes the preprocessing of genome sequencing data and the application of a GATK pipeline (Best Practices Workflow) for variant analysis. Using sequencing data for chromosome 11 from individuals HG00149 to HG00178 sourced from the 1000 Genomes Project, the pipeline replicated a standard analysis workflow with industry-standard tools based on the GATK best practices manual [1]. The analysis employed GATK4 for variant calling and recalibration, SAMtools for preprocessing, and BWA for sequence alignment.

The initial genomes aligned to GRCh37 were realigned to GRCh38 to ensure compatibility with GATK4 tools and references. Outputs include recalibrated BAM files, a joint VCF, and key statistics such as the transition/transversion (Ti/Tv) ratio and average indel length. Challenges encountered, including read mismatches and unpaired reads, were effectively resolved to produce high-quality results. This project provided valuable hands-on experience in genomic data processing and analysis using modern bioinformatics tools.

1 Materials and Tools

Data

Samples: The sequencing data used in this study was obtained from the 1000 Genomes Project and consisted of chromosome 11 sequencing data for the following individuals: HG00149, HG00150, HG00151, HG00154, HG00155, HG00157, HG00158, HG00159, HG00160, HG00171, HG00173, HG00174, HG00176, HG00177, and HG00178. The total dataset size was approximately 7.28 GB.

Files

- **Chromosome 11 Alignment BAM Files:**
  - These are `.bam` files containing aligned sequencing reads specific to chromosome 11 for each sample.
  - **Example:** `HG00149.chrom11.ILLUMINA.bwa.GBR.exome.20130415.bam`
  - **Format:** BAM (Binary Alignment/Map) is a compressed, binary representation of sequencing reads aligned to a reference genome. Each BAM file contains:
    - **Header information**: Metadata including reference genome details, sequencing platform, and processing details.
    - **Aligned reads**: Information on chromosome, position, sequence, quality scores, and alignment flags.
- **BAM Index Files:**

- Corresponding `.bai` files provide an index for the BAM files, enabling efficient processing and quick access to specific genomic regions.
- **Example:** `HG00149.chrom11.ILLUMINA.bwa.GBR.exome.20130415.bam.bai`

Reference Genomes

- **GRCh37/hg19**: The original reference genome used for initial BAM alignments.
- **GRCh38/hg38**: The updated reference genome used for re-alignment and variant calling. Aligning to GRCh38 ensured compatibility with the tools needed for the workflow, particularly GATK4.

The GRCh38 reference genome was downloaded using the following commands:

```
wget -P /mnt/hdd_b/home/user02/reference/hg38/ \
    https://storage.googleapis.com/genomics-public-data/resources/broad/hg38
       ↪ /v0/hg38.fa.gz
wget -P /mnt/hdd_b/home/user02/reference/hg38/ \
    https://storage.googleapis.com/genomics-public-data/resources/broad/hg38
       ↪ /v0/hg38.fa.gz.fai
wget -P /mnt/hdd_b/home/user02/reference/hg38/ \
    https://storage.googleapis.com/genomics-public-data/resources/broad/hg38
       ↪ /v0/hg38.dict
```

We downloaded the reference genome along with its index and its dictionary. Detailed instructions for downloading and verifying the reference genome are documented in the log file `download_data.txt`(Appendix).

External Resources

To ensure accurate variant calling and recalibration, high-confidence variant datasets were incorporated into the pipeline according to the instructions given in the workflow manual.

Mills and 1000G Gold Standard INDELs and HapMap Datasets

These resources contain well-characterized variant sites that help refine variant quality filtering and recalibration.

The required datasets were downloaded from the Broad Institute's public genomic resources using the following commands:

```
# Mills and 1000G Gold Standard INDELs
wget -P "$output_dir" https://storage.googleapis.com/genomics-public-data/
   ↪ resources/broad/hg38/v0/Mills_and_1000G_gold_standard.indels.hg38.vcf.
   ↪ gz
```

```
wget -P "$output_dir" https://storage.googleapis.com/genomics-public-data/resources/broad/hg38/v0/Mills_and_1000G_gold_standard.indels.hg38.vcf.gz.tbi

# HapMap SNP dataset
wget -P "$output_dir" https://storage.googleapis.com/genomics-public-data/resources/broad/hg38/v0/hapmap_3.3.hg38.vcf.gz
wget -P "$output_dir" https://storage.googleapis.com/genomics-public-data/resources/broad/hg38/v0/hapmap_3.3.hg38.vcf.gz.tbi
```

These datasets provide a benchmark for variant recalibration, ensuring that only high-confidence variants are retained in the final call set. Additional details on how these resources were utilized can be found in the pipeline documentation and in Appendix.

2 Software and Tools

Bioinformatics Tools

The following software tools were used throughout the analysis pipeline:

- **GATK (Genome Analysis Toolkit)**: Version 4.6.1.0 was used for variant calling, recalibration, and data processing. The tool was downloaded from the official Broad Institute GATK website.
- **SAMtools**: Used for BAM file manipulation, sorting, and indexing.
- **BWA (Burrows-Wheeler Aligner)**: Used for sequence alignment to the hg38 reference genome.
- **BCFTools**: Used for statistical analysis of variant data and filtering.
- **Picard**: Used for adding read groups and marking duplicates in BAM files.
- **Python and Jupyter Notebook**: Used to generate plots of allele frequency distributions, depth of coverage, and quality scores.

File Download Process

Sequencing data for chromosome 11 from individuals HG00149 to HG00178 (excluding HG00154 due to missing data) were obtained from the 1000 Genomes Project on European Bioinformatics Institute website. The `.bam` files, which contain aligned sequencing reads specific to chromosome 11, and their corresponding `.bai` index files were downloaded using `wget`.

This process was not done manually for each sample but automated using a shell script (`download_data.sh`) that loops over all the sample IDs to ensure all relevant files were retrieved

efficiently. Details of this process are included in the Appendix. The necessary input files, including BAM files and reference genome resources, were obtained using the scripts found in the appendix.

Installing GATK

GATK version 4.6.1.0 can be downloaded and extracted using the following commands:

```
# Download GATK version 4.6.1.0
wget -P "$output_dir" https://github.com/broadinstitute/gatk/releases/
    ↪ download/4.6.1.0/gatk-4.6.1.0.zip

# Unzip the downloaded file
unzip "$output_dir/gatk-4.6.1.0.zip" -d "$output_dir"
```

3 Pipeline

Preprocessing

Conversion to FASTQ

The original BAM files, aligned to the GRCh37 reference genome, were converted into paired-end FASTQ files to prepare for realignment to the GRCh38 reference genome. This step was performed using `samtools fastq`:

```
$SAMTOOLS fastq \
 -1 $SAMPLE_OUTPUT_DIR / $ { SAMPLE } _R1 . fastq \
 -2 $SAMPLE_OUTPUT_DIR / $ { SAMPLE } _R2 . fastq \
 -s $SAMPLE_OUTPUT_DIR / $ { SAMPLE } _singletons . fastq \
 -0 / dev / null \
 -n $BAM_FILE
```

Key Parameters:

- **-1 and -2**: Output files for paired-end reads.
- **-s**: Outputs singleton reads (reads without a pair).
- **-n**: Ensures consistent naming of paired-end reads.
- **-0 /dev/null**: Discards unmapped reads.

Alignment to GRCh38

FASTQ files were aligned to the GRCh38 reference genome (specifically chromosome 11) using `bwa mem`. The alignment step was performed as follows:

```
$BWA mem -t 4 $BWA_INDEX \
 $SAMPLE_OUTPUT_DIR / $ { SAMPLE } _R1 . fastq \
 $SAMPLE_OUTPUT_DIR / $ { SAMPLE } _R2 . fastq > $SAMPLE_OUTPUT_DIR / $ {
    ↪ SAMPLE
,     }. sam
```

Key Parameters:

- **-t 4**: Utilized 4 threads to optimize performance.
- **Input FASTQ files**: Paired-end reads generated in the previous step.
- **Output SAM file**: Contains the alignments against the GRCh38 reference genome.
- **Files Produced**: SAM files (`.sam`) containing aligned reads for each sample.

Sorting and Indexing

The SAM files generated from the alignment step were converted to BAM format, sorted by coordinates, and indexed for downstream processing. This was performed using the following commands:

Convert SAM to BAM and Sort:

```
$SAMTOOLS view - bS $SAMPLE_OUTPUT_DIR / $ { SAMPLE }. sam | \
 $SAMTOOLS sort -o $SAMPLE_OUTPUT_DIR / $ { SAMPLE } _sorted . bam
```

Index BAM File:

```
$SAMTOOLS index $SAMPLE_OUTPUT_DIR / $ { SAMPLE } _sorted . bam
```

Files Produced:

- Sorted BAM files (`_sorted.bam`) for each sample.
- BAM index files (`_sorted.bam.bai`) for efficient downstream access.

The entire preprocessing workflow was automated using a custom shell script, `realign_data.sh`, which processed all samples in the dataset. The script includes steps for BAM-to-FASTQ conversion, alignment to GRCh38, and sorting/indexing, ensuring consistency and efficiency. Full details of the script are available in the Appendix.

Results

The alignment quality for each sample was assessed using `samtools flagstat`. For sample HG00149 as example, the following metrics were obtained:

- **Total Reads**: 199,461 reads (including QC-passed and failed reads).
- **Mapped Reads**: 196,844 reads (98.69% of total reads).
- **Properly Paired Reads**: 190,946 reads (95.78% of paired reads).
- **Singleton Reads**: 2,601 reads (1.30%).
- **Duplicates**: No duplicate reads identified.

These results demonstrate high-quality alignments, with the majority of reads successfully mapped and properly paired.

Variant Calling with HaplotypeCaller

Purpose

The objective of this step was to detect genetic variants (SNPs and INDELs) from the processed BAM files and generate GVCF (Genomic Variant Call Format) files. The GVCF format provides both variant and non-variant information, enabling efficient joint genotyping in subsequent steps.

Files Produced

- **GVCF Files**: A `.g.vcf.gz` file was generated for each sample. This file contains variant and non-variant regions, which will later be combined for joint genotyping.

Script Used

The following shell script was used to process all BAM files and generate GVCF files for each sample:

```
for bam_file in /mnt/hdd_b/home/user02/output/*/*_sorted.bam
do
    sample_name=$(basename $bam_file _sorted.bam)
    mkdir -p /mnt/hdd_b/home/user02/output/$sample_name
    gatk --java-options "-Xmx4G" HaplotypeCaller \
        -R /mnt/hdd_b/home/user02/reference/hg38/hg38_chrom11.fa \
        -I $bam_file \
        -O /mnt/hdd_b/home/user02/output/$sample_name/${sample_name}.g.vcf.
           ↪ gz \
```

```
        -ERC GVCF
done
```

Key Parameters:

- **-Xmx4G**: Allocates **4GB** of memory for Java execution.
- **-R**: Specifies the reference genome file ($/\texttt{mnt/hdd}_b/home/user02/reference/hg38/hg38\_chrom11.fa$ $InputBAMfilecontainingthealignedsequencingreads.$
- **-O**: Output GVCF file where variant calls will be stored (`$sample_name.g.vcf.gz`).
- **-ERC GVCF**: Enables **GVCF mode**, which records both variant and non-variant sites for later joint genotyping.

Quality Control and Recalibration

Purpose

This step ensures that the BAM files are of high quality for downstream variant calling. It includes the following:

1. Adding missing read group information.
2. Identifying and marking duplicate reads to prevent biases.
3. Recalibrating base quality scores (BQSR) to adjust for systematic errors.

Read Group Addition

Using `Picard AddOrReplaceReadGroups`, missing read group information was added to each BAM file. Read groups allow proper handling of samples during variant calling and help tools like GATK distinguish between different sequencing runs.

MarkDuplicates

Duplicate reads were identified and marked using GATK's `MarkDuplicates` tool to avoid biases during variant calling. This step helps prevent PCR amplification artifacts from affecting variant calls.

Base Quality Score Recalibration (BQSR)

To address systematic biases in base quality scores, recalibration was performed using GATK `BaseRecalibrator` and `ApplyBQSR`. This process corrects for sequencing errors that vary based on sequencing cycle and nucleotide context, improving overall variant calling accuracy.

The specific commands used for `MarkDuplicates` and `BQSR` are provided in the shell script `duplicates_bqsr.sh` (Appendix).

Consolidation with GenomicsDBImport

Purpose

In this step we consolidated the individual GVCF files generated for each sample into a GenomicsDB workspace. This GDB workspace allows efficient storage and management of multi-sample data, enabling streamlined joint genotyping in the next step.

Creating the Sample Map File

Before running `GenomicsDBImport`, a sample map file was required to link each sample name to its respective GVCF file path. This was automated using the `create_sample_map.sh` script.

Explanation:

- The script dynamically identified the `.g.vcf.gz` files for each sample (HG00149 to HG00178) and generated a `sample_map.txt` file.
- The `sample_map.txt` file lists each sample name and the corresponding GVCF file path separated by a single space(critical for GenomicsDB to work as we encountered an error multiple times).

Command Used for GenomicsDB

The following command was used to import the data into the GenomicsDB workspace:

```
gatk GenomicsDBImport \
   -R /mnt/hdd_b/home/user02/reference/hg38/hg38.fa \
   --sample-name-map /mnt/hdd_b/home/user02/sample_map.txt \
   --genomicsdb-workspace-path /mnt/hdd_b/home/user02/output/genomicsdb \
   --intervals chr11 \
   --batch-size 50
```

Explanation of Parameters

- **-R**: Specifies the reference genome (GRCh38).
- –**sample-name-map**: The sample map file linking sample names to their respective GVCF files.
- –**genomicsdb-workspace-path**: The directory where the GenomicsDB workspace is created.
- –**intervals**: Specifies chromosome 11 for targeted analysis.
- –**batch-size**: The number of samples to process simultaneously. Setting this to 50 optimizes memory usage.

Runtime

The `GenomicsDBImport` step completed successfully in **3.66 minutes**, consolidating data from all samples into the database.

Output

- **GenomicsDB Workspace**: A workspace was created containing the consolidated data from all 15 samples.
- **Supporting files within the workspace**:
  - `vidmap.json`
  - `callset.json`
  - `vcfheader.vcf`

Joint Genotyping with GenotypeGVCFs

Purpose

After importing data into the GenomicsDB workspace, the next step is joint-calling the cohort. This process uses GATK's `GenotypeGVCFs` tool to process the consolidated GenomicsDB and produce a joint variant callset in VCF format.

Command Used

The following command was used for joint genotyping:

```
gatk GenotypeGVCFs \
    -R /mnt/hdd_b/home/user02/reference/hg38/hg38.fa \
    -V gendb:///mnt/hdd_b/home/user02/output/genomicsdb \
    -O /mnt/hdd_b/home/user02/output/chr11_joint.vcf.gz
```

Explanation of Parameters

- **-R**: Specifies the reference genome (GRCh38).
- **-V**: Specifies the input data source (the GenomicsDB workspace).
- **-O**: Specifies the output file for the joint VCF.

Results

- **Elapsed Time**: The joint-calling process took **9.26 minutes** to process 10,628,775 variants from chromosome 11 across the 15 samples.
- **Output File**: The final joint VCF file was generated `chr11_joint.vcf.gz`
- **Performance**: The process maintained a consistent speed of approximately 1,150,000 variants per minute, demonstrating efficient hardware and software performance.

VCF Validation with ValidateVariants

Purpose

The final joint VCF file was validated using GATK's `ValidateVariants` tool to ensure it was properly formatted and free of errors. This step confirms that the VCF is ready for downstream analysis.

Command Used

The following command was used to validate the VCF:

```
gatk ValidateVariants \
    -R /mnt/hdd_b/home/user02/reference/hg38/hg38.fa \
    -V /mnt/hdd_b/home/user02/output/chr11_joint.vcf.gz
```

Results

- **Variants Processed**: The tool processed **73,805 variants** in the `chr11_joint.vcf.gz` file.
- **Elapsed Time**: The validation process completed in **0.04 minutes**, indicating a highly efficient operation.
- **Outcome**: The VCF file is valid and ready for downstream analysis.

Filter Variants by Quality Score Recalibration

Purpose

This step refines the joint VCF file by applying Variant Quality Score Recalibration (VQSR). VQSR uses machine learning models trained on known, high-confidence datasets such as HapMap and Mills to distinguish between high-confidence and low-confidence variants. This process improves variant calling accuracy by filtering out potentially false positives.

Building the Recalibration Model

The `VariantRecalibrator` tool was used to create separate recalibration models for SNPs and INDELs.

Commands for SNPs

```
gatk VariantRecalibrator \
    -R /mnt/hdd_b/home/user02/reference/hg38/hg38.fa \
    -V /mnt/hdd_b/home/user02/output/chr11_joint.vcf.gz \
    --resource:hapmap,known=false,training=true,truth=true,prior=15.0 /mnt/
       ↪ hdd_b/home/user02/reference/hg38/hapmap_3.3.hg38.vcf.gz \
    --resource:omni,known=false,training=true,truth=true,prior=12.0 /mnt/
       ↪ hdd_b/home/user02/reference/hg38/1000G_omni2.5.hg38.vcf.gz \
    --resource:1000G,known=false,training=true,truth=false,prior=10.0 /mnt/
       ↪ hdd_b/home/user02/reference/hg38/1000G_phase1.snps.high_confidence
       ↪ .hg38.vcf.gz \
    -an QD -an MQ -an FS -an MQRankSum -an ReadPosRankSum -an SOR \
    -mode SNP \
    -O /mnt/hdd_b/home/user02/output/recalibrate_SNP.recal \
    --tranches-file /mnt/hdd_b/home/user02/output/recalibrate_SNP.tranches \
    --rscript-file /mnt/hdd_b/home/user02/output/recalibrate_SNP_plots.R
```

Commands for INDELs

```
gatk VariantRecalibrator \
    -R /mnt/hdd_b/home/user02/reference/hg38/hg38.fa \
    -V /mnt/hdd_b/home/user02/output/chr11_joint.vcf.gz \
    --resource:hapmap,known=false,training=true,truth=true,prior=15.0 /mnt/
       ↪ hdd_b/home/user02/reference/hg38/hapmap_3.3.hg38.vcf.gz \
    --resource:omni,known=false,training=true,truth=true,prior=12.0 /mnt/
       ↪ hdd_b/home/user02/reference/hg38/1000G_omni2.5.hg38.vcf.gz \
    --resource:1000G,known=false,training=true,truth=false,prior=10.0 /mnt/
       ↪ hdd_b/home/user02/reference/hg38/1000G_phase1.snps.high_confidence
       ↪ .hg38.vcf.gz \
    -an QD -an MQ -an FS -an MQRankSum -an ReadPosRankSum -an SOR \
    -mode INDEL \
    -O /mnt/hdd_b/home/user02/output/recalibrate_INDEL.recal \
    --tranches-file /mnt/hdd_b/home/user02/output/recalibrate_INDEL.tranches
       ↪  \
    --rscript-file /mnt/hdd_b/home/user02/output/recalibrate_INDEL_plots.R
```

Key Parameters:

- **-R**: Specifies the **reference genome** (GRCh38), which is required to correctly interpret variant positions.

- **-V**: Specifies the **input VCF file** containing the joint-called variants for recalibration.
- –**resource:hapmap**: Uses the **HapMap dataset** as a high-confidence known variant resource for SNP training, with a prior probability of 15.0.
- –**resource:omni**: Uses the **1000G Omni dataset** as another trusted resource for variant recalibration, with a prior probability of 12.0.
- –**resource:1000G**: Uses the **1000 Genomes high-confidence SNPs** as a training set but not as a truth set, with a prior probability of 10.0.
- **-an QD**: Uses **Quality by Depth** (QD) as an annotation feature in the model.
- **-an MQ**: Uses **Mapping Quality** (MQ) as an annotation to evaluate read mapping accuracy.
- **-an FS**: Uses **Fisher Strand Bias** (FS) to measure strand bias in sequencing reads.
- **-an MQRankSum**: Uses the **MQRankSum** statistic to compare mapping quality between reference and alternate alleles.
- **-an ReadPosRankSum**: Uses the **Read Position Rank Sum** test to check for systematic differences in read position between reference and alternate alleles.
- **-an SOR**: Uses the **Strand Odds Ratio** (SOR) as an improved measure of strand bias.
- **-mode SNP**: Specifies that the recalibration model is being built for **SNPs** (not INDELs).
- **-O**: Outputs the recalibration table as `recalibrate_SNP.recal`.
- –**tranches-file**: Saves the **tranches file**, which defines confidence levels for filtering.
- –**rscript-file**: Generates an R script (`recalibrate_SNP_plots.R`) to visualize recalibration performance.

Applying the Recalibration Model

Once the recalibration model was built, it was applied to filter SNPs and INDELs in the joint VCF file.

Command for SNPs

```
gatk ApplyVQSR \
    -R /mnt/hdd_b/home/user02/reference/hg38/hg38.fa \
    -V /mnt/hdd_b/home/user02/output/chr11_joint.vcf.gz \
    -O /mnt/hdd_b/home/user02/output/chr11_joint_filtered_SNPs.vcf.gz \
    --recal-file /mnt/hdd_b/home/user02/output/recalibrate_SNP.recal \
    --tranches-file /mnt/hdd_b/home/user02/output/recalibrate_SNP.tranches \
    -mode SNP
```

Command for INDELs

```
gatk ApplyVQSR \
    -R /mnt/hdd_b/home/user02/reference/hg38/hg38.fa \
    -V /mnt/hdd_b/home/user02/output/chr11_joint_filtered_SNPs.vcf.gz \
    -O /mnt/hdd_b/home/user02/output/chr11_joint_filtered.vcf.gz \
    --recal-file /mnt/hdd_b/home/user02/output/recalibrate_INDEL.recal \
    --tranches-file /mnt/hdd_b/home/user02/output/recalibrate_INDEL.tranches
       ↪  \
    -mode INDEL
```

Key Parameters

- –**resource**: Reference datasets (`HapMap`, `Mills`, etc.) used to train the recalibration model.
- **-an**: Annotations like `QD` (Quality by Depth), `FS` (Fisher Strand Bias), and `MQ` (Mapping Quality) used for modeling.
- **-mode**: Specifies whether the command targets **SNPs** or **INDELs**.
- –**tranches-file**: Defines confidence levels for filtering.
- –**recal-file**: Input recalibration file generated during model building.

Results

- **Filtered VCF File:** The final filtered VCF file was generated: `chr11_joint_filtered.vcf.gz`
- **Recalibration Plots:** Diagnostic plots (`recalibrate_SNP_plots.R` and `recalibrate_INDEL_plots`) were generated to visualize the model's performance. However, an issue was encountered with the color parameters in the R script:
  - The script attempted to use the RGB color space, which was incompatible with the installed version of the R graphics package.

- The error arose because the plotting library expected Lab color space parameters instead of RGB.
- The R scripts were manually modified to replace RGB parameters with Lab color space parameters, ensuring compatibility with the graphics package.
- After this adjustment, the plots were successfully generated, including metrics such as the recalibration model's performance and tranche thresholds for SNPs and INDELs.

Statistical Analysis

Purpose

After filtering variants using VQSR, a detailed statistical analysis of the filtered VCF file was conducted to evaluate the quality and distribution of the data. This analysis included:

1. Basic metrics for variant types (SNPs, INDELs, etc.).
2. Transition/transversion (Ti/Tv) ratio.
3. Allele frequency distribution.
4. Depth of coverage (DP).
5. Quality score (QUAL) statistics.
6. Indel analysis (count and length).

Commands and Results

1. Basic Variant Metrics Using `bcftools stats`, the following metrics were extracted from the filtered VCF file:

```
bcftools stats /mnt/hdd_b/home/user02/output/chr11_joint_filtered.vcf.gz >
    ↪ filtered_stats.txt
grep "^SN" filtered_stats.txt
```

Results:

- Number of Samples: **15**
- Total Variants: **73,805**
- SNPs: **68,528**
- INDELs: **5,279**

- Multi-allelic Sites: **64**
- Multi-allelic SNP Sites: **15**

2. Transition/Transversion (Ti/Tv) Ratio The Ti/Tv ratio was calculated using `bcftools stats`:

```
bcftools stats /mnt/hdd_b/home/user02/output/chr11_joint_filtered_strict.vcf
    ↪ .gz | grep TSTV
```

Results:

- Transitions (ts): **43,308**
- Transversions (tv): **25,235**
- Ti/Tv Ratio: **1.72**

3. Allele Frequency Distribution Allele frequencies were extracted with `bcftools`:

```
bcftools +fill-tags /mnt/hdd_b/home/user02/output/
    ↪ chr11_joint_filtered_strict.vcf.gz -- -t AF | \
grep -v "^#" | awk '{print $8}' | cut -d";" -f1 | cut -d"=" -f2 >
    ↪ allele_frequencies.txt
```

Results (Python Statistics):

- Mean: **2.22**
- Standard Deviation: **0.91**
- Rare Variants (Frequency ¡ 0.05): **0**

4. Depth of Coverage (DP) Depth of coverage was queried for all variants:

```
bcftools query -f '%CHROM\t%POS\t%INFO/DP\n' /mnt/hdd_b/home/user02/output/
    ↪ chr11_joint_filtered_strict.vcf.gz > depth.txt
```

Results (Python Statistics):

- Mean: **4.77**
- Median: **4.00**
- Low Coverage Sites (Depth ¡ 10): **71,020**

5. Quality Scores (QUAL) Quality scores were queried:

```
bcftools query -f '%CHROM\t%POS\t%QUAL\n' /mnt/hdd_b/home/user02/output/
    ↪ chr11_joint_filtered_strict.vcf.gz > quality_scores.txt
```

Results (Python Statistics):

- Mean: **68.76**
- Median: **51.05**
- Maximum: **24,788.5**

6. Indel Analysis Using `VariantsToTable`, INDEL metrics were extracted:

```
gatk VariantsToTable \
    -V /mnt/hdd_b/home/user02/output/chr11_joint_filtered_strict.vcf.gz \
    -O indels_table.txt \
    -F CHROM -F POS -F REF -F ALT -F TYPE
```

Counting Insertions and Deletions:

```
awk '{if ($5 == "INDEL" && length($4) > length($3)) print $0}' indels_table.
    ↪ txt | wc -l  # Insertions
awk '{if ($5 == "INDEL" && length($3) > length($4)) print $0}' indels_table.
    ↪ txt | wc -l  # Deletions
```

Results:

- Insertions: **2,139**
- Deletions: **2,840**

Average Indel Length Calculation:

```
awk '{if ($5 == "INDEL") print length($4)-length($3)}' indels_table.txt | \
awk '{sum+=($1>=0?$1:-$1)} END {print "Average Indel Length:", sum/NR}'
```

Result:

- Average Indel Length: **2.17 bases**

Generated Plots

The following plots were created for data visualization:

- **Allele Frequencies:** Distribution of allele frequencies.
- **Depth of Coverage:** Distribution of sequencing depth.
- **Quality Scores:** Distribution of variant quality scores.

Allele Frequency Distribution

Interpretation:

- The majority of variants have an allele frequency of 2, indicating that they are common within this sample cohort.
- A few variants show higher frequencies, but they are significantly less frequent.
- The presence of no rare variants (frequency ¡ 0.05) confirms that most detected variants are shared among the samples.
- This distribution suggests a dataset largely composed of common polymorphisms rather than rare mutations.

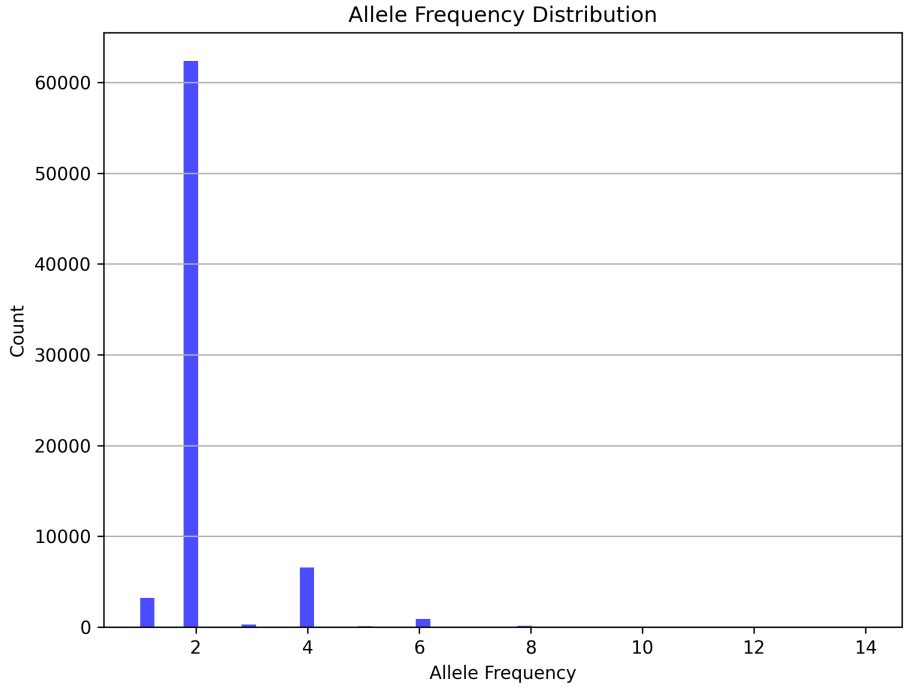

Figure 1: Allele Frequency Distribution

Quality Score Distribution

Interpretation:

- The majority of variants have a quality score between 40 and 100, with a peak around 50.
- A smaller fraction of variants has much higher quality scores, extending beyond 200.
- The long tail in the distribution suggests a small number of variants with exceptionally high confidence.
- This is expected in a well-filtered dataset, as lower-quality variants would have been removed during the VQSR filtering step.

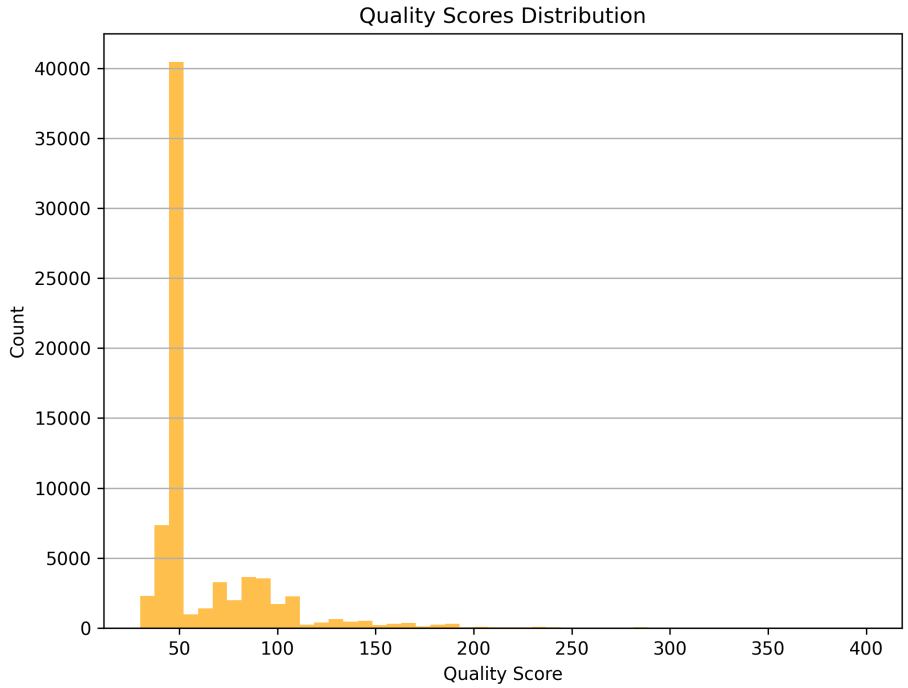

Figure 2: Quality Score Distribution

Depth of Coverage Distribution

Interpretation:

- The majority of variants have a sequencing depth between 3 and 7, with a peak around 4.

- There are a small number of variants with much higher coverage, but they are relatively rare.
- The low coverage of most sites (with a mean depth of 4.77) is expected given that this is an exome capture dataset, where sequencing is targeted rather than whole-genome.
- Sites with extremely high depth are likely due to repetitive regions or collapsed alignments, which should be considered when interpreting variant confidence.

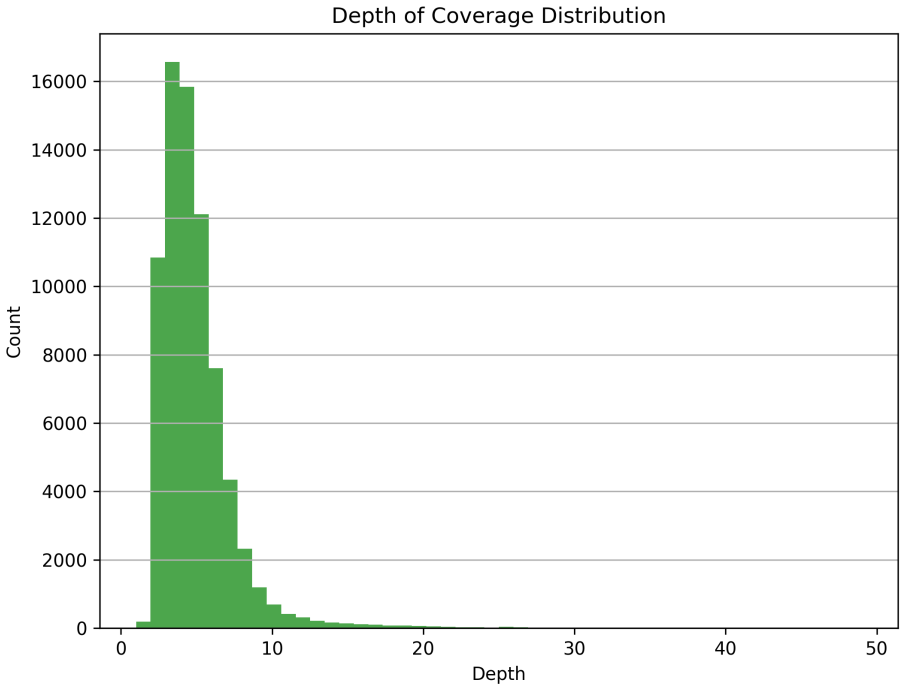

Figure 3: Depth of Coverage Distribution

Summary of Results

These visualizations confirm that:

1. Most variants are common SNPs, with a dominant allele frequency of 2.
2. Quality scores are generally high, supporting confidence in the variant calls.
3. Depth of coverage is mostly low but follows an expected exome sequencing pattern.

These results validate the filtering steps performed earlier and confirm that the final VCF file is high-quality and well-curated for downstream analysis.

4 Discussion

Results

The final filtered VCF file contained **73,805 variants**, with **68,528 SNPs** and **5,279 INDELs**. The transition/transversion (Ti/Tv) ratio was **1.72**, which falls within the expected range for human exome sequencing data.

The allele frequency distribution showed that most variants were shared among samples, with a dominant frequency of **2**, indicating common polymorphisms. The quality scores were generally high, with a mean of **68.76**, ensuring confidence in variant calls. The depth of coverage was relatively low, with a mean of **4.77**, which aligns with the exome sequencing approach.

Data Quality

The high quality scores and expected Ti/Tv ratio suggest that the variant filtering process was effective, removing low-confidence calls while retaining reliable variants. The depth of coverage distribution shows a skew toward lower depths, which is typical for targeted sequencing but could indicate that some regions may have insufficient coverage for confident variant calling.

The absence of rare variants ($\mathbf{AF < 0.05}$) suggests that either the cohort lacks rare polymorphisms or that the variant calling process was biased toward common alleles.

Challenges Encountered

- A technical issue was encountered during the **Variant Quality Score Recalibration (VQSR)** step, where the R script used for recalibration plots produced errors due to an **RGB vs. Lab color space mismatch**. This required manually editing the script to replace RGB parameters with Lab color parameters for proper plot generation.

- Due to the storage limitations of our virtual server, we encountered issues where intermediate files accumulated and caused the system to run out of disk space, leading to process failures. To mitigate this, we had to implement a strategy of deleting intermediate files immediately after they were no longer needed, ensuring that the pipeline could complete successfully without interruptions.

Limitations of the Analysis

- **Low coverage at many sites**: With a mean depth of **4.77**, some genomic regions may not have sufficient sequencing depth for confident variant calling.

- **Lack of rare variants**: The absence of variants with an allele frequency ¡ **0.05** suggests either a cohort with highly similar genetic backgrounds or possible filtering biases against rare alleles.
- **Exome sequencing limitations**: The data only includes **chromosome 11 exonic regions**, meaning structural variations and deep intronic mutations are not detected.

Future Improvements

- **Higher coverage sequencing**: Increasing the depth of coverage could improve confidence in variant calls, particularly for rare variants.
- **Incorporation of additional variant databases**: Comparing results with `dbSNP`, `ClinVar`, and `gnomAD` could help refine variant interpretation.
- **Refining filtering thresholds**: Adjusting the **VQSR tranche settings** or applying additional manual filtering could help capture more biologically relevant variants.
- **Computational optimizations**: Running the pipeline in a **high-performance computing (HPC)** environment could reduce processing times and enable scaling for larger cohorts.

Overall, the workflow effectively identified and filtered **high-confidence variants**, providing a reliable dataset for downstream genetic analysis. Further refinements could enhance sensitivity and accuracy, particularly in detecting rare or low-frequency variants.

5 Conclusion

We successfully implemented the GATK pipeline for variant discovery and filtering in **chromosome 11 exonic regions** across the samples. The workflow included **preprocessing, alignment, variant calling, quality filtering, and statistical analysis**, ensuring the generation of high-confidence variants.

Key findings:

- Most detected variants were **common SNPs**, with a **Ti/Tv ratio of 1.72** and high overall quality scores.
- Depth of coverage was **low** but consistent with expectations for targeted exome sequencing.
- The **allele frequency distribution** indicated a **lack of rare variants**, which could be due to sample selection or filtering stringency.

Challenges such as **computational resource demands** and **plotting errors in VQSR** were successfully resolved, allowing smooth execution of the workflow.

Overall, this analysis provided valuable hands-on experience with modern bioinformatics tools such as `GATK`, `BWA`, `Samtools`, and `BCFTools`, offering practical insights into genomic data processing and interpretation.

6 Appendix

Scripts Used

`download_data.sh`

```
#!/bin/bash

FTP_BASE_PATH="ftp://ftp.1000genomes.ebi.ac.uk/vol1/ftp/phase3/data"
LOCAL_BASE_PATH="./data"

START_SAMPLE=149
END_SAMPLE=178

mkdir -p "$LOCAL_BASE_PATH"

for i in $(seq -w $START_SAMPLE $END_SAMPLE); do
    SAMPLE="HG00${i}"
    REMOTE_PATH="${FTP_BASE_PATH}/${SAMPLE}/exome_alignment/"
    LOCAL_PATH="${LOCAL_BASE_PATH}/${SAMPLE}/exome_alignment/"

    mkdir -p "$LOCAL_PATH"

    if curl --silent --head --fail "${REMOTE_PATH}" > /dev/null; then
        echo "Processing ${SAMPLE}..."

        FILES=$(curl -s "${REMOTE_PATH}" | grep "chrom11")

        for FILE in $FILES; do
            wget -q --show-progress --no-parent --cut-dirs=6 -P "$LOCAL_PATH
               ↪ " "${REMOTE_PATH}${FILE}"
        done
    else
        echo "Directory ${REMOTE_PATH} does not exist, skipping"
    fi
done

echo "Completed"
```

download_reference.sh

```
#!/bin/bash

output_dir="/mnt/hdd_b/home/user02/reference/hg38/"

mkdir -p "$output_dir"

# Mills and 1000G Gold Standard INDELs
wget -P "$output_dir" https://storage.googleapis.com/genomics-public-data/resources/broad/hg38/v0/Mills_and_1000G_gold_standard.indels.hg38.vcf.gz
wget -P "$output_dir" https://storage.googleapis.com/genomics-public-data/resources/broad/hg38/v0/Mills_and_1000G_gold_standard.indels.hg38.vcf.gz.tbi

# HapMap
wget -P "$output_dir" https://storage.googleapis.com/genomics-public-data/resources/broad/hg38/v0/hapmap_3.3.hg38.vcf.gz
wget -P "$output_dir" https://storage.googleapis.com/genomics-public-data/resources/broad/hg38/v0/hapmap_3.3.hg38.vcf.gz.tbi

echo "Downloads completed. Files are saved in $output_dir"
```

realign_data.sh

```
#!/bin/bash

REFERENCE="/mnt/hdd_b/home/user02/reference/hg38/hg38_chrom11.fa"
DATA_DIR="/mnt/hdd_b/home/user02/data/"
OUTPUT_DIR="/mnt/hdd_b/home/user02/output/"
BWA_INDEX="$REFERENCE"

SAMTOOLS="/usr/bin/samtools"
BWA="/usr/bin/bwa"

for BAM_FILE in $DATA_DIR/HG*/exome_alignment/*.bam; do
    SAMPLE=$(basename $BAM_FILE .bam)

    SAMPLE_OUTPUT_DIR="$OUTPUT_DIR/$SAMPLE"
    mkdir -p $SAMPLE_OUTPUT_DIR

    echo "Converting BAM to FASTQ for $SAMPLE..."
    $SAMTOOLS fastq \
        -1 $SAMPLE_OUTPUT_DIR/${SAMPLE}_R1.fastq \
        -2 $SAMPLE_OUTPUT_DIR/${SAMPLE}_R2.fastq \
        -s $SAMPLE_OUTPUT_DIR/${SAMPLE}_singletons.fastq \
```

```
        -0 /dev/null \
        -n $BAM_FILE

    echo "Aligning FASTQ for $SAMPLE to reference genome (chr11)..."
    $BWA mem -t 4 $BWA_INDEX \
        $SAMPLE_OUTPUT_DIR/${SAMPLE}_R1.fastq \
        $SAMPLE_OUTPUT_DIR/${SAMPLE}_R2.fastq > $SAMPLE_OUTPUT_DIR/${SAMPLE
            ↪ }.sam

    echo "Converting and sorting SAM to BAM for $SAMPLE..."
    $SAMTOOLS view -bS $SAMPLE_OUTPUT_DIR/${SAMPLE}.sam | \
        $SAMTOOLS sort -o $SAMPLE_OUTPUT_DIR/${SAMPLE}_sorted.bam

    echo "Indexing BAM for $SAMPLE..."
    $SAMTOOLS index $SAMPLE_OUTPUT_DIR/${SAMPLE}_sorted.bam

    echo "Processing completed for $SAMPLE."
done

echo "All samples processed."
```

duplicates_bqsr.sh

```
#!/bin/bash

REFERENCE="/mnt/hdd_b/home/user02/reference/hg38/hg38_chrom11.fa"
KNOWN_SITES_MILLS="/mnt/hdd_b/home/user02/reference/hg38/
    ↪ Mills_and_1000G_gold_standard.indels.hg38.vcf.gz"
KNOWN_SITES_HAPMAP="/mnt/hdd_b/home/user02/reference/hg38/hapmap_3.3.hg38.
    ↪ vcf.gz"
OUTPUT_DIR="/mnt/hdd_b/home/user02/output"
TOOLS_DIR="/mnt/hdd_b/home/user02/tools/gatk-4.6.1.0/gatk"
PICARD_JAR="/mnt/hdd_b/home/user02/reference/hg38/picard.jar"

for BAM in $OUTPUT_DIR/*/*_sorted.bam; do
    SAMPLE_NAME=$(basename $BAM _sorted.bam)
    SAMPLE_DIR=$(dirname $BAM)

    echo "Processing sample: $SAMPLE_NAME"

    # Add Read Groups if missing
    java -jar $PICARD_JAR AddOrReplaceReadGroups \
        I=$BAM \
        O=$SAMPLE_DIR/${SAMPLE_NAME}_with_RG.bam \
        RGID=$SAMPLE_NAME \
        RGLB=lib1 \
        RGPL=ILLUMINA \
```

```
        RGPU=unit1 \
        RGSM=$SAMPLE_NAME

    # Mark Duplicates
    gatk MarkDuplicates \
        -I $SAMPLE_DIR/${SAMPLE_NAME}_with_RG.bam \
        -O $SAMPLE_DIR/${SAMPLE_NAME}_marked_duplicates.bam \
        -M $SAMPLE_DIR/${SAMPLE_NAME}_marked_dup_metrics.txt \
        --CREATE_INDEX true

    # Base Recalibration
    gatk BaseRecalibrator \
        -R $REFERENCE \
        -I $SAMPLE_DIR/${SAMPLE_NAME}_marked_duplicates.bam \
        --known-sites $KNOWN_SITES_MILLS \
        --known-sites $KNOWN_SITES_HAPMAP \
        -O $SAMPLE_DIR/${SAMPLE_NAME}_recal_data.table

    echo "Completed processing for $SAMPLE_NAME."
done

echo "All samples processed successfully."
```

Python Script for Plotting (bioinfoplotting.ipynb)

```
import pandas as pd
import matplotlib.pyplot as plt

allele_frequencies_path = "allele_frequencies.txt"
quality_scores_path = "quality_scores.txt"
depth_path = "depth.txt"

allele_freqs = pd.read_csv(allele_frequencies_path, sep="\t", header=None,
    ↪ names=["Frequency"])
quality_scores = pd.read_csv(quality_scores_path, sep="\t", header=None,
    ↪ names=["Quality"])
depth = pd.read_csv(depth_path, sep="\t", header=None, names=["Chromosome",
    ↪ "Position", "Depth"])

# Quality Scores Histogram
plt.figure(figsize=(8, 6))
plt.hist(quality_scores["Quality"], bins=50, color='orange', alpha=0.7)
plt.title("Quality Scores Distribution")
plt.xlabel("Quality Score")
plt.ylabel("Count")
plt.grid(axis='y')
plt.savefig("QualityScore.png", dpi=300)
```

```
plt.show()

# Allele Frequency Distribution
plt.figure(figsize=(8, 6))
plt.hist(allele_freqs["Frequency"], bins=50, color='blue', alpha=0.7)
plt.title("Allele Frequency Distribution")
plt.xlabel("Allele Frequency")
plt.ylabel("Count")
plt.grid(axis='y')
plt.savefig("allele_freq.png", dpi=300)
plt.show()

# Depth of Coverage Distribution
plt.figure(figsize=(8, 6))
plt.hist(depth["Depth"], bins=50, color='green', alpha=0.7)
plt.title("Depth of Coverage Distribution")
plt.xlabel("Depth")
plt.ylabel("Count")
plt.grid(axis='y')
plt.savefig("depth.png", dpi=300)
plt.show()
```

6.1 Files present in the folder

Scripts

1. `create_sample_map.sh` – Script to generate a sample map for GenomicsDBImport.
2. `download_data.sh` – Script to download the BAM files from 1000 Genomes FTP.
3. `download_reference.sh` – Script to download reference datasets (Mills, HapMap).
4. `duplicates_bqsr.sh` – Script to mark duplicates and perform Base Quality Score Recalibration (BQSR).
5. `haplo.sh` – Script for running GATK HaplotypeCaller.
6. `realign_data.sh` – Script for realigning the sequencing reads.

Data Files

7. `allele_frequencies.txt` – Extracted allele frequencies from the filtered VCF.
8. `depth.txt` – Sequencing depth values for each variant position.
9. `filtered_stats.txt` – General statistics for the filtered VCF file.

10. `HG00149_stats.txt` – Specific statistics for the HG00149 sample.
11. `indels_table.txt` – Table containing information on insertions and deletions.
12. `pipeline.txt` – Documentation of the pipeline steps.
13. `quality_scores.txt` – Extracted variant quality scores from the VCF.
14. `raw_stats.txt` – Statistics of the raw (unfiltered) VCF file.
15. `download_data.txt` – Documentation of the data download process.
16. `sample_map.txt` – Mapping of samples
17. `statistics.txt` – Documentation of the statistical analysis.

Images & Plots

18. `allele_freq.png` – Plot of the allele frequency distribution.
19. `depth.png` – Plot of the sequencing depth distribution.
20. `QualityScore.png` – Plot of the quality scores distribution.

Jupyter Notebook

21. `bioinfoplotting.ipynb` – Jupyter notebook used to generate plots and statistics.

References

[1] Broad Institute. *Germline short variant discovery (SNPs + Indels).* Available at: `https://gatk.broadinstitute.org/hc/en-us/articles/360035535932-Germline-short-variant-discovery-SNPs-Indels`

[2] Broad Institute. *Genome Analysis Toolkit (GATK) v4.6.1.* Available at: `https://github.com/broadinstitute/gatk`

[3] Li, H. et al. *The Sequence Alignment/Map (SAM) format and SAMtools.* Bioinformatics, 2009. Available at: `http://www.htslib.org/`

[4] Li, H. *Burrows-Wheeler Aligner (BWA).* Bioinformatics, 2009. Available at: `http://bio-bwa.sourceforge.net/`

[5] Danecek, P. et al. *BCFTools - Variant calling and manipulation.* Available at: `http://samtools.github.io/bcftools/`

[6] 1000 Genomes Project Consortium. *A map of human genome variation from population-scale sequencing.* Nature, 2010. Available at: `https://www.internationalgenome.org/`